Ensayo LLVM

Desarrollar esta tarea de OLC1 fue básicamente poner a prueba todo lo que vimos en clase, pero ya en la vida real, armando un compilador desde cero usando C, Flex, Bison y LLVM. La meta principal era simple de decir, pero compleja de hacer: agarrar código fuente, validar que estuviera bien escrito y sacar instrucciones listas para correr en una arquitectura específica.

Para arrancar, nos tocó usar Flex con el análisis léxico para ir detectando cada token (palabras clave, identificadores, números, operadores, etc.). De ahí pasamos a Bison para el análisis sintáctico y armar el árbol de sintaxis abstracta (AST). Esto último fue clave porque nos dio una estructura ordenada para procesar las instrucciones sin perdernos en el intento.

Luego vino el análisis semántico con la tabla de símbolos. Aquí fue donde revisamos que las variables existieran antes de usarlas y que los tipos fueran compatibles en las operaciones. Es curioso (y frustrante a veces) darte cuenta de que un código puede pasar perfecto la sintaxis, pero tronar por completo en la semántica.

Y bueno, lo que fue un verdadero dolor de cabeza fue instalar y configurar todas las herramientas necesarias para que funcionaran bien juntas en el entorno. Entre dependencias, versiones y paths, más de uno se llevó un buen rato solo logrando que compilaran las librerías base.

Ya librado eso, la parte central fue usar LLVM para sacar el código intermedio (LLVM IR). A partir del AST armamos la lógica para aritmética, variables y salidas. Lo chido de LLVM es que te abstrae un montón; nosotros lo usamos para generar código ARM directo para el procesador Cortex-M0+ de la Raspberry Pi Pico, e incluso integramos una instrucción para encender el LED de la placa y sacar el firmware en `.uf2`.

Durante todo el proceso nos topamos con mil trabas entre la generación de código y la integración con el hardware. A pesar de eso, a punta de pruebas y corrección de bugs, logramos pasar las 117 pruebas automatizadas y sacar el firmware, aunque nos faltó probarlo físicamente en la placa.

En conclusión, este proyecto te cambia la perspectiva de cómo funciona un compilador por dentro, desde que entra el texto plano hasta que baje a código de máquina. Entendes de verdad el peso de cada etapa y por qué herramientas como Flex, Bison y LLVM son tan salvavidas, siempre y cuando tengas paciencia y organización para armar el rompecabezas.